

三、程序框架

詹文翰 高级实验师

zhanwenhan@163. com

正向工程

导出程序框架

软件开发环境

软件开发环境是指用于进行软件开发的工具、平台和设置的组合。它提供了一个开发者可以编写、测试和分析代码的环境。其通常包括：

1. 编程语言和框架：编程语言是开发者用来编写软件的规则和语法。框架是一组提供了特定功能和结构的代码库，简化了开发过程。
2. 开发和调试工具：包括集成开发环境、文本编辑器、编译器、链接器、调试器等。
3. 版本控制系统：用于跟踪和管理代码的不同版本。
4. 测试和分析工具：测试工具用于自动化测试和验证软件的功能和性能。分析工具用于评估和优化软件的性能，识别性能瓶颈和优化机会。
5. 集成和部署工具：集成工具用于将不同的开发工具和系统集成在一起，实现自动化的构建、测试和部署流程。部署工具用于将软件应用程序部署到目标环境中，例如服务器、云平台等。

不同的开发者通常会根据自己的偏好和项目要求选择适合的软件开发环境。

常见测试和分析工具

白盒测试框架：gtest、boosttest

Googletest（通常简称为gtest）是由Google开发的一个C++单元测试框架，用于编写和运行自动化测试。它支持各种测试方法，包括单元测试、集成测试和功能测试等，并提供了丰富的测试工具和测试报告，以帮助开发人员快速、准确地测试和验证代码的正确性和健壮性。

内存测试工具：valgrind

Valgrind是一套Linux下，开放源代码的仿真调试工具的集合。Valgrind由内核以及基于内核的其他调试工具组成。内核模拟了一个CPU环境，并提供服务给其他工具；而其他工具类似于插件，利用内核提供的服务完成各种特定的内存调试任务。

性能测试工具：gprof、perf

gprof(GNU profiler)是GNU binutils工具集中的一个工具，Linux系统当中会自带这个工具。它可以分析程序的性能，能给出函数调用时间、调用次数和调用关系，找出程序的瓶颈所在。在编译和链接选项中都加入-pg之后，gcc会在每个函数中插入代码片段，用于记录函数间的调用关系和调用次数，并采集函数的调用时间。

统一开发环境

软件开发环境各不相同，选择合适的环境即可。但是，对于项目协作来说，软件开发环境**务必统一**，以防止出现不必要的麻烦。

开发环境不一致导致的问题：

1. 代码可移植性失效，频繁出现“本地通过、别处编译失败”
2. 二进制不兼容引发链接错误与运行时崩溃
3. 依赖管理混乱，版本与参数漂移
4. 运行环境差异导致的程序行为不一致
5. 调试与问题复现难度陡增

统一开发环境的方法（推荐）：

1. 远程共享同一套软件开发环境
2. 容器/虚拟机镜像复制
3. 编写统一环境配置脚本（Infrastructure as Code）

软件编码规范

除软件开发环境需要统一外，制定并统一软件开发规范同样重要：

1. 统一代码风格，便于代码审查和维护；
2. 降低出现程序Bug的风险；
3. 养成良好的编程习惯；
4. 进一步学习语言特性；

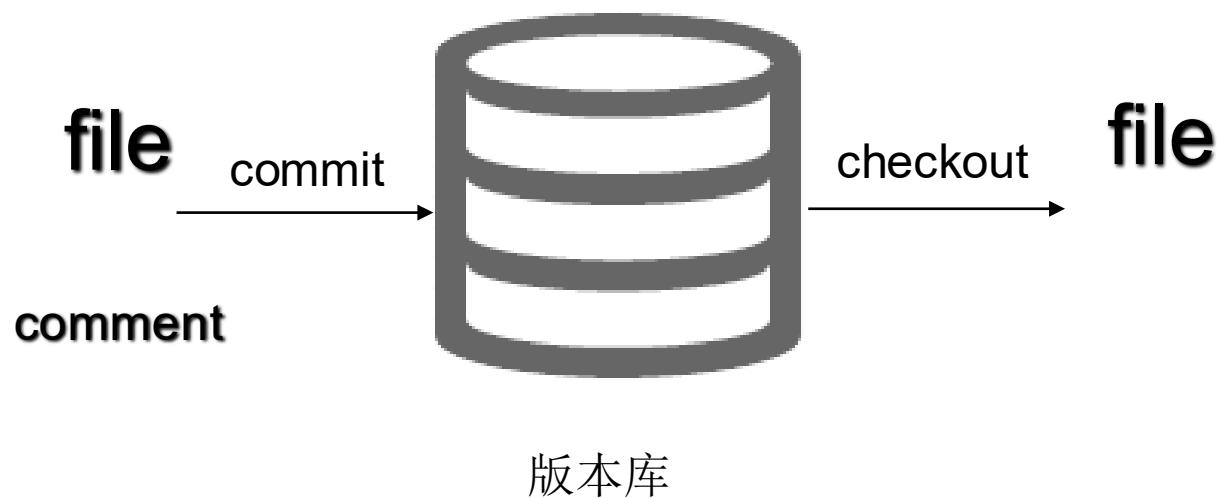
编码规范实例：

[Google - 开源项目风格指南](#)

华为 - 软件编程规范

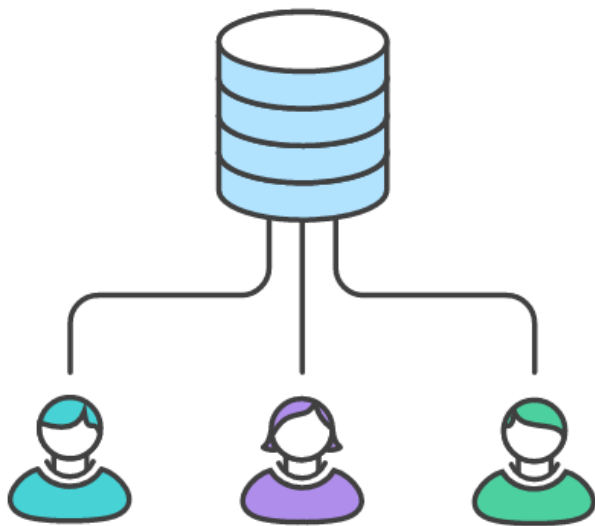
版本控制

版本控制软件也被称为代码时光机。有了版本库，代码任何更改都会保存起来，并且我们可以对每一次更改提交添加评论（**comment**），说明更改的内容或目的。



版本控制

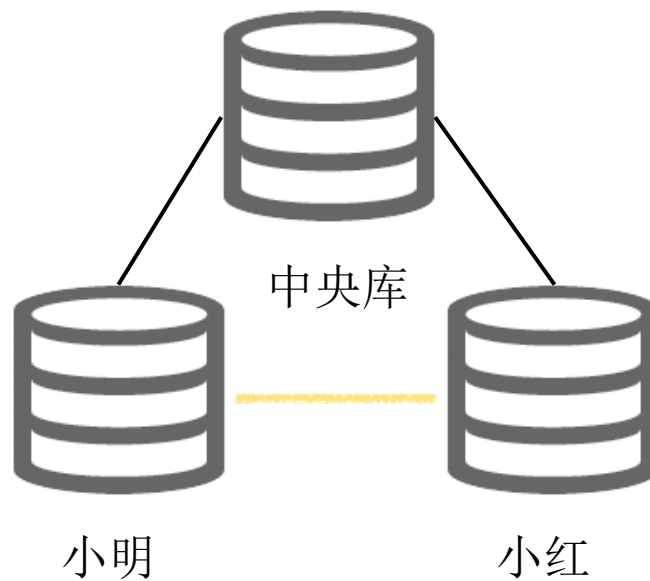
版本控制系统分类：



集中式：如SVN、CVS

做什么事都需要与中央库进行交互，包括查看日志。

缺点：单点失效

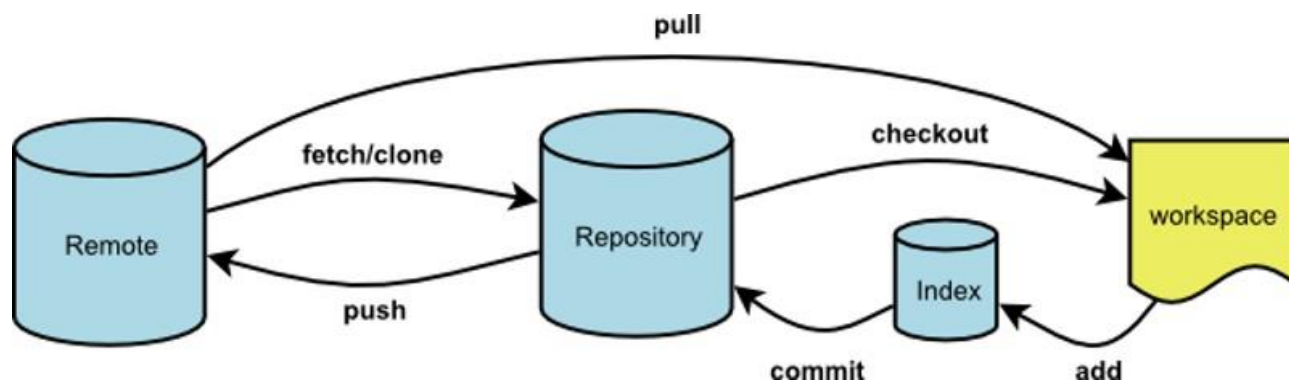


分布式：Git

只需要第一次与中央库进行交互（克隆版本库），之后只需要单机提交到本地版本库。在合适的时候，再与“中央库”进行合并。

版本控制软件——git

git init
git remote add origin git@github.com:帐号名/仓库名.git
git clone git@github.com:帐号名/仓库名.git
git add
git commit -m
git push origin master
git pull origin master
git fetch origin
git checkout
git rm
git branch
git status
git log
.....



版本控制基本概念

版本

每一次正确的代码提交，会为版本库生成一个版本，所有版本可以基于依赖关系构成一条代码演进的时间线（树）。

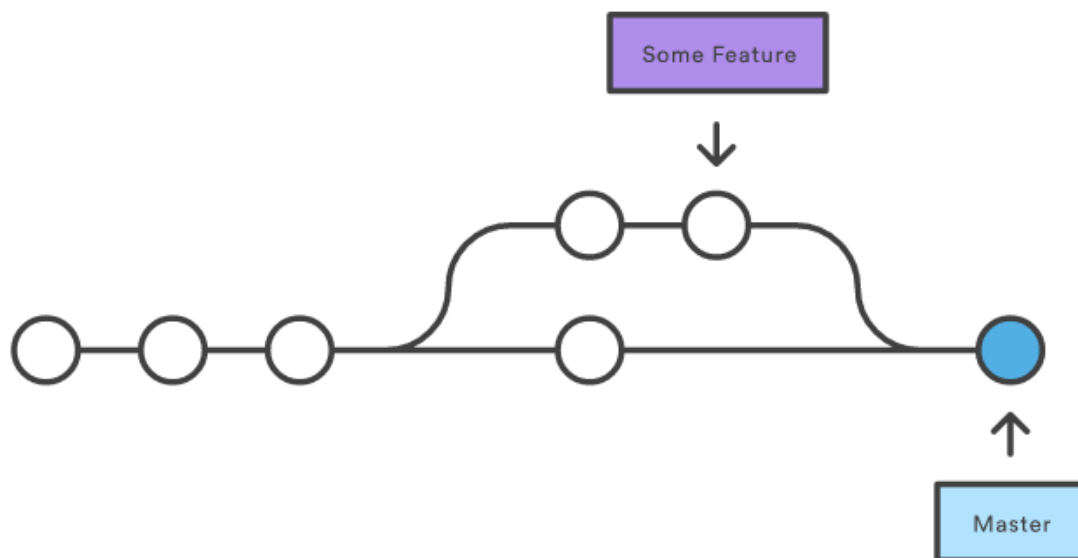
分支

使用分支意味着可以把你的工作从开发主线上分离开来，以免影响开发主线。分支可以根据不同的目的进行划分。

分支分类

1) 以功能开发为导向

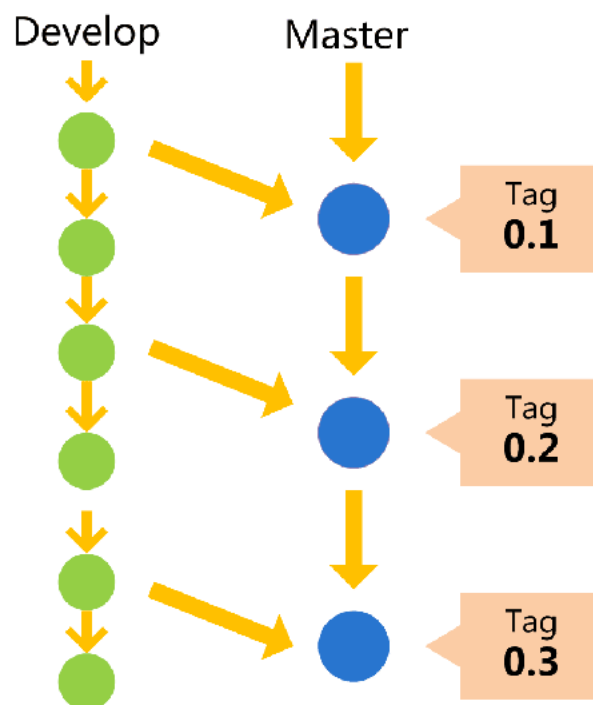
基于功能增加来完成迭代的项目，产生多个分支的目的是为了单独开发某一功能。最终为了实现统一的整体，需要在功能开发完成之后进行分支合并（merge）。



分支分类

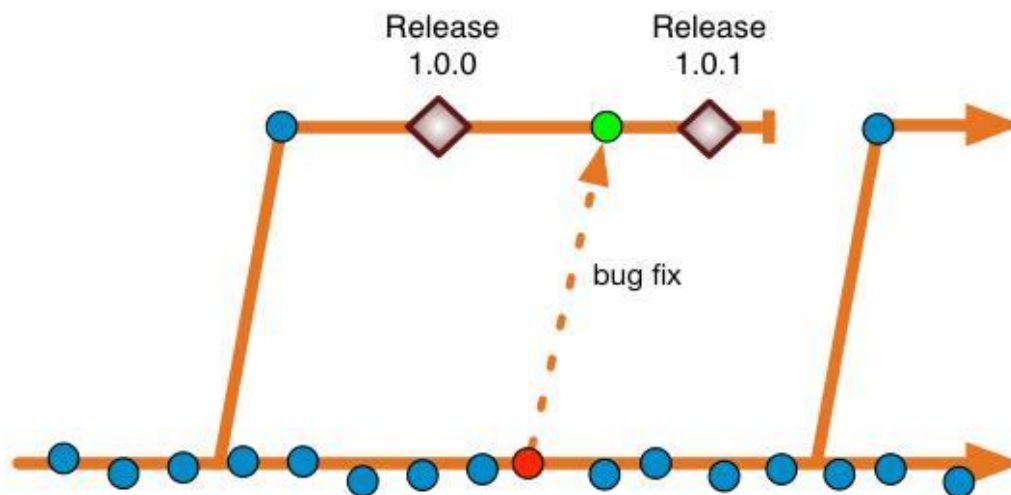
2) 以项目管理为导向

我们可以将分支设置为多个：如用于给用户提供稳定的软件版本 **master** 主分支；用于项目开发的 **develop** 分支；用于软件发布准备的 **release** 分支；用于系统维护、修复bug的 **hotfix** 分支等。



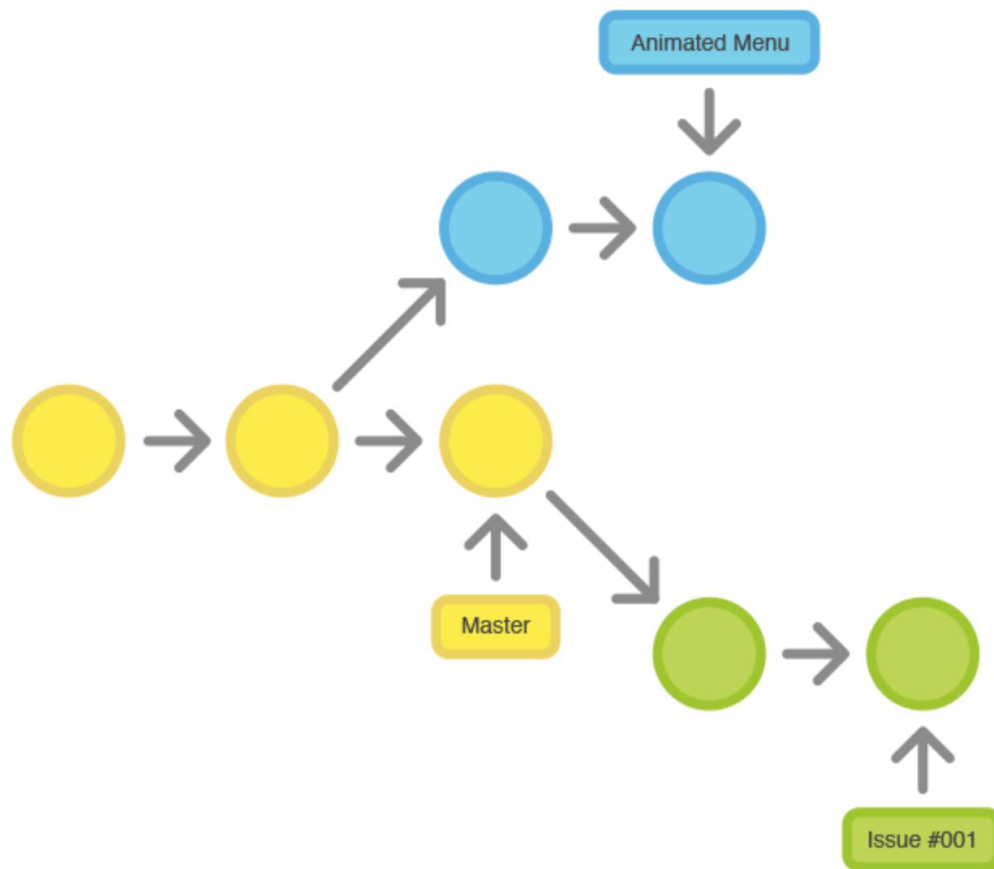
软件协作 workflow

1、集中式 workflow (trunk workflow)



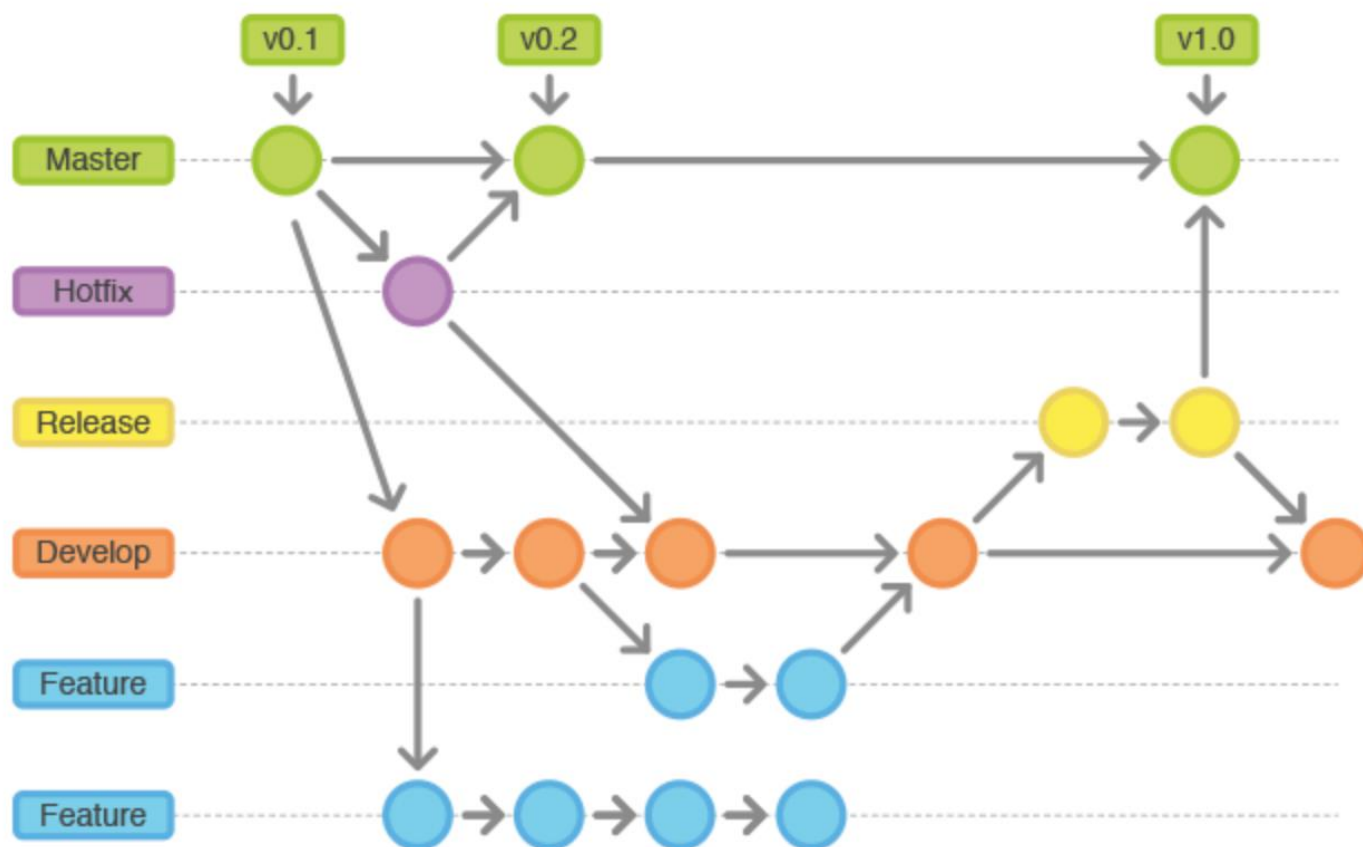
软件协作 workflow

2、功能分支 workflow



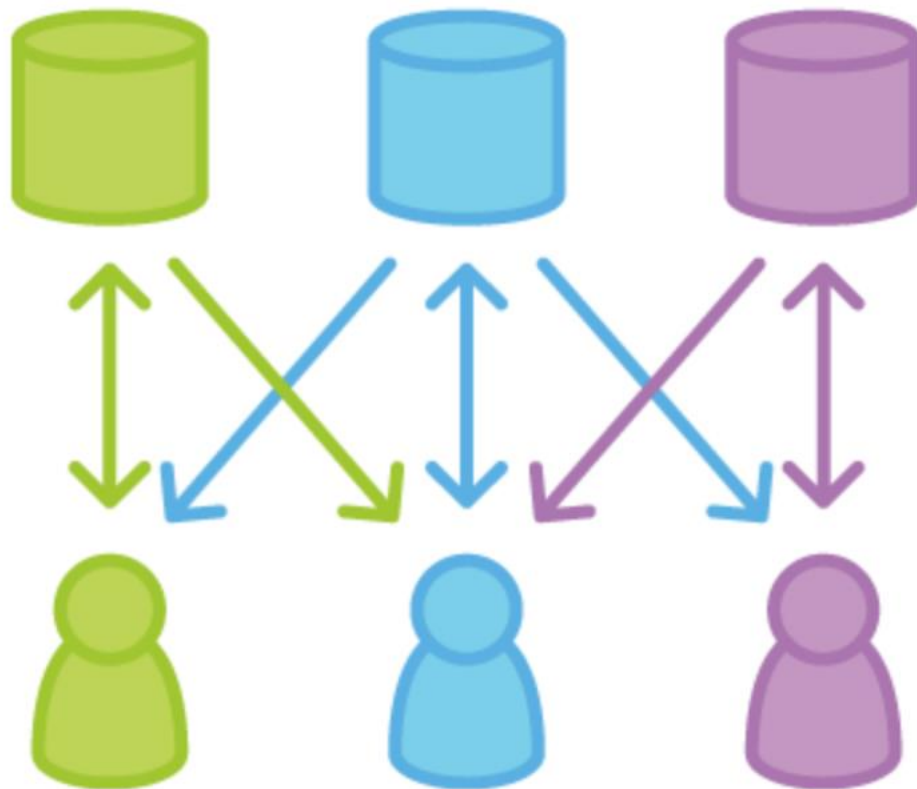
软件协作 workflow

3、gitflow workflow



软件协作 workflow

4、forking workflow



实验环节

完成以下实验内容：

- 完成实验指导书1（软件开发环境）中指定练习（选修）
 - 无需提交实验报告
- 完成实验指导书2（软件开发协作）中指定练习
 - 无需提交实验报告
- 完成本课程项目的代码框架编写
- 实现多文件编译，保证代码框架编译通过
 - 小组采用统一软件开发环境进行项目开发
 - 使用git进行项目协作开发
 - 采用Github/gitee创建在线版本库
 - 选定合适的软件协作工作流
 - 在项目文档中对自身的软件开发环境进行详细介绍，包括：
 - 开发平台
 - 编程语言
 - 构建工具
 - 调试工具
 - 第三方库
 - 版本控制系统
 - 性能分析工具
 - 集成和部署工具